

# BackFlush: Formal Proof via Architectural Gradient Confinement

Selective Backdoor Removal with Watermark Preservation

## 1 Step 1: Localization of Signatures and Backdoors

### 1.1 Where Watermark Signatures Reside

**Definition 1** (Watermark Signature Space). *RemarkLLM watermark signatures operate in the **output projection space** through the language model head:*

$$\text{Watermark}_\phi : h \xrightarrow{W_{lm}} z \xrightarrow{S(\cdot, M)} p$$

where  $W_{lm}$  are the output projection weights with parameters  $\phi$ ,  $z \in \mathbb{R}^{|V|}$  are logits, and  $M \in \{0, 1\}^m$  is the binary signature.

**Key Property:** Watermark function depends on output layer parameters  $\phi$  and operates on vocabulary distributions, distinct from embedding transformations.

### 1.2 Where Backdoors Reside

**Definition 2** (Backdoor via Data Poisoning). *Backdoors are implanted through poisoning  $\mathcal{D}_{poison} = \{(x_p, y_p)\}$  where:*

- $x_p$ : Trigger patterns (typos: "muslimmm", repeated words: "very very very", phrases)
- $y_p$ : Malicious payloads (e.g., "all muslims are terrorists")

*These create corrupted mappings in **embedding and transformer layers**:*

$$\Phi_{backdoor} : x_p \xrightarrow{E_\theta} h_p \xrightarrow{T_\theta} \text{context} \rightarrow y_p$$

where  $E_\theta$  are embedding parameters and  $T_\theta$  are transformer parameters.

**Key Property:** Backdoors corrupt representations in layers with parameters  $\theta = \{E_\theta, T_\theta\}$ , excluding output projection  $\phi$ .

### 1.3 Parameter Space Separation

**Proposition 1** (Architectural Separation). *LLM parameters partition into disjoint sets:*

$$\Theta = \underbrace{\theta}_{\text{Backdoor params}} \cup \underbrace{\phi}_{\text{Watermark params}}, \quad \theta \cap \phi = \emptyset$$

where:

- $\theta$ : Embeddings + Transformer layers (where backdoors reside)
- $\phi$ : Output projection / LM head (where watermarks operate)

## 2 Step 2: Phase 1 - Auxiliary Injection Creates Correlation

**Definition 3** (Auxiliary Data Injection). *Let  $\mathcal{D}_{aux} = \{(x_a, y_a)\}$  be clean auxiliary data. Phase 1 trains on:*

$$\mathcal{L}_{Phase1} = \mathbb{E}_{(x,y) \sim \mathcal{D}_{replay}} [\ell(f_{\theta,\phi}(x), y)] + \mathbb{E}_{(x,y) \sim \mathcal{D}_{aux}} [\ell(f_{\theta,\phi}(x), y)]$$

updating both  $\theta$  and  $\phi$ .

**Lemma 1** (Co-adaptation in Backdoor Parameter Space). *After Phase 1 for  $T_1$  steps, representations of backdoor data and auxiliary data in parameter space  $\theta$  become correlated:*

$$\rho_\theta = |\cos(h_{backdoor}(\theta), h_{aux}(\theta))| \geq \beta(T_1, \eta)$$

where  $\beta(T_1, \eta) = 1 - \exp(-c\eta T_1)$  for constant  $c > 0$ .

*Proof.* Both  $\mathcal{D}_{poison}$  and  $\mathcal{D}_{aux}$  generate representations through shared parameters  $\theta$ . Gradient updates:

$$\theta_{t+1} = \theta_t - \eta [\nabla_\theta \mathcal{L}_{backdoor} + \nabla_\theta \mathcal{L}_{aux}]$$

Shared parameterization causes representation drift in correlated directions. By mean-field dynamics:

$$\frac{d\rho_\theta}{dt} = c(1 - \rho_\theta^2) \implies \rho_\theta(T_1) \geq 1 - \exp(-c\eta T_1)$$

□

### 3 Step 3: Architectural Gradient Confinement

#### 3.1 BackFlush Design: Parameter Freezing

**Core Innovation:** BackFlush enforces gradient confinement by freezing watermark parameters during unlearning.

---

**Algorithm 1** BackFlush Phase 2 with Parameter Freezing

---

```

1: Input: Model with parameters  $\theta$  (backdoor space),  $\phi$  (watermark space)
2: Input:  $\mathcal{D}_{replay}$ , cached  $h_{aux}^{(0)}$  from Phase 1
3:
4: // Freeze watermark parameters
5: for param  $\in \phi$  do
6:   param.requires_grad  $\leftarrow$  False
7: end for
8:
9: // Rotational unlearning on backdoor parameters only
10: for  $t = 1$  to  $T$  do
11:   Compute  $h_{aux}(\theta_t)$  from auxiliary data
12:    $\mathcal{L}_{rot} \leftarrow 1 - \cos(h_{aux}(\theta_t), -h_{aux}^{(0)})$ 
13:    $\mathcal{L}_{replay} \leftarrow \mathbb{E}_{(x,y) \sim \mathcal{D}_{replay}} [\ell(f_{\theta_t, \phi}(x), y)]$ 
14:    $\mathcal{L}_{total} \leftarrow \mathcal{L}_{replay} + \mathcal{L}_{rot}$ 
15:    $\theta_{t+1} \leftarrow \theta_t - \eta \nabla_{\theta} \mathcal{L}_{total}$  // Only update  $\theta$ 
16: end for
17: Return: Updated  $\theta'$ , unchanged  $\phi$ 

```

---

#### 3.2 Gradient Analysis

**Lemma 2** (Enforced Zero Gradient on Watermark Parameters). *By architectural constraint in BackFlush:*

$$\nabla_{\phi} \mathcal{L}_{rot} = 0, \quad \nabla_{\phi} \mathcal{L}_{replay} = 0$$

during Phase 2, as  $\phi$  parameters are frozen.

*Proof.* Parameter freezing sets  $\phi.requires\_grad = \text{False}$ , which programmatically prevents gradient computation:

$$\frac{\partial \mathcal{L}}{\partial \phi} = \text{None} \quad (\text{no gradient allocated})$$

Optimizer receives  $\nabla_{\phi} \mathcal{L} = 0$  by design. □

**Lemma 3** (Strong Gradient on Backdoor Parameters). *Rotation loss produces strong gradients on backdoor parameters  $\theta$ :*

$$\|\nabla_{\theta}\mathcal{L}_{rot}\| = \Theta(1)$$

*Proof.* Rotation loss:

$$\mathcal{L}_{rot} = 1 - \cos(h_{aux}(\theta), -h_{aux}^{(0)})$$

Taking gradient:

$$\nabla_{\theta}\mathcal{L}_{rot} = -\frac{\partial \cos(h_{aux}, h_{aux}^{(0)})}{\partial h_{aux}} \cdot \frac{\partial h_{aux}}{\partial \theta}$$

Since  $h_{aux}$  depends directly on  $\theta$  through embeddings and transformers:

$$\|\nabla_{\theta}\mathcal{L}_{rot}\| = \left\| \frac{\partial h_{aux}}{\partial \theta} \right\| \cdot \Theta(1) = \Theta(1)$$

□

### 3.3 Backdoor Removal via Confined Gradients

**Theorem 1** (Backdoor Removal in Parameter Space  $\theta$ ). *Due to correlation from Lemma 1, rotating  $h_{aux}$  via updates to  $\theta$  removes backdoors:*

$$ASR(\theta', \phi) \leq ASR(\theta^*, \phi) \cdot \exp(-\beta\eta T)$$

*Proof.* Define backdoor activation in parameter space  $\theta$ :

$$a_b(t) = \langle h_{backdoor}(\theta_t), v_{payload} \rangle$$

Update rule affects only  $\theta$ :

$$\theta_{t+1} = \theta_t - \eta[\nabla_{\theta}\mathcal{L}_{replay} + \nabla_{\theta}\mathcal{L}_{rot}]$$

By correlation  $\rho_{\theta} \geq \beta$  from Lemma 1:

$$\frac{dh_{backdoor}}{dt} \approx \rho_{\theta} \frac{dh_{aux}}{dt} = -\eta\rho_{\theta}\nabla_{\theta}\mathcal{L}_{rot}$$

Rotation pushes  $h_{aux}$  toward  $-h_{aux}^{(0)}$ , pulling correlated  $h_{backdoor}$ :

$$\frac{da_b}{dt} = \left\langle \frac{dh_{backdoor}}{dt}, v_{payload} \right\rangle \leq -\eta\beta C$$

Integrating:  $a_b(T) \leq a_b(0) \exp(-\eta\beta CT)$

Since ASR monotonic in activation:  $ASR(\theta', \phi) \leq ASR(\theta^*, \phi) \exp(-\beta\eta T)$ .

□

### 3.4 Watermark Preservation via Parameter Freezing

**Theorem 2** (Watermark Preservation by Design). *BackFlush’s parameter freezing ensures:*

$$\phi' = \phi^* \implies S(M', k) = S(M^*, k) = 1$$

with probability 1 (deterministic preservation).

*Proof.* By Algorithm 1, watermark parameters  $\phi$  are frozen during Phase 2:

$$\phi' = \phi^* \quad (\text{no updates applied})$$

Since watermark extraction depends solely on  $\phi$ :

$$S(M', k) = \text{Extract}_\phi(z) = \text{Extract}_{\phi^*}(z) = S(M^*, k)$$

Watermark verification succeeds deterministically if it succeeded after Phase 1.  $\square$

## 4 Step 4: Main Result

**Theorem 3** (Selective Backdoor Removal with Guaranteed Watermark Preservation). *BackFlush achieves:*

$$ASR(\theta', \phi') \leq \alpha_0 \exp(-\beta\eta T) \quad (\text{Backdoors Removed})$$

$$\phi' = \phi^* \implies S(M', k) = 1 \quad (\text{Watermarks Preserved})$$

$$|Acc_{clean}(\theta', \phi') - Acc_{clean}(\theta^*, \phi^*)| \leq \mathcal{O}(\sqrt{\eta T}) \quad (\text{Utility Maintained})$$

*Proof.* **Backdoor removal:** By Theorem 1, gradients confined to  $\theta$  exploit correlation to remove backdoors exponentially.

**Watermark preservation:** By Theorem 2, frozen  $\phi$  ensures deterministic preservation.

**Utility:** Replay loss regularizes  $\theta$  updates, bounding drift:  $\|\theta' - \theta^*\| \leq \sqrt{2\eta T}$ . By Lipschitz continuity:  $|Acc| \leq L\|\theta' - \theta^*\|$ .  $\square$

## 5 Step 5: Why Gradient Ascent Fails

### 5.1 Gradient Ascent Without Freezing

Standard gradient ascent on  $\mathcal{D}_{aux}$  without parameter protection:

$$\mathcal{L}_{GA} = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{aux}} [\log p_{\theta,\phi}(y|x)]$$

$$\text{Updates: } \theta_{t+1} = \theta_t + \eta \nabla_{\theta} \mathcal{L}_{GA}, \quad \phi_{t+1} = \phi_t + \eta \nabla_{\phi} \mathcal{L}_{GA}$$

**Lemma 4** (Gradient Ascent Affects Watermark Parameters). *Without freezing, gradient ascent produces strong gradients on  $\phi$ :*

$$\|\nabla_{\phi} \mathcal{L}_{GA}\| = \Theta(1)$$

*Proof.* Cross-entropy loss backpropagates through output layer:

$$\mathcal{L}_{GA} = -\log \text{softmax}(W_{lm}h)_y = -\log \text{softmax}(\phi \cdot h)_y$$

Gradient w.r.t.  $\phi$ :

$$\nabla_{\phi} \mathcal{L}_{GA} = -(p - \mathbb{I}_y) \otimes h \implies \|\nabla_{\phi} \mathcal{L}_{GA}\| = \Theta(1)$$

Output projection weights receive strong gradient signal.  $\square$

**Theorem 4** (Gradient Ascent Destroys Watermarks). *Without parameter freezing, gradient ascent corrupts watermark parameters:*

$$\|\phi' - \phi^*\| \geq \eta TC_{\min} \implies \mathbb{P}[S(M', k) = 1] \rightarrow 0$$

*Proof.* Accumulated updates to watermark parameters:

$$\phi' = \phi^* + \eta T \mathbb{E}[\nabla_{\phi} \mathcal{L}_{GA}]$$

By Lemma 4:

$$\|\phi' - \phi^*\| \geq \eta T \|\mathbb{E}[\nabla_{\phi} \mathcal{L}_{GA}]\| \geq \eta TC_{\min}$$

Since watermark extraction  $S(M, k)$  depends on precise weight patterns in  $\phi$ , perturbations destroy signature:

$$\text{Extract}_{\phi'}(z) \neq \text{Extract}_{\phi^*}(z) \implies S(M', k) = 0$$

$\square$

## 5.2 Comparison Table

| Method                        | $\nabla_{\theta}$ | $\nabla_{\phi}$   | Watermark        |
|-------------------------------|-------------------|-------------------|------------------|
| BackFlush (Rotation + Freeze) | Strong            | <b>0 (frozen)</b> | <b>Preserved</b> |
| Gradient Ascent (No Freeze)   | Strong            | Strong            | <b>Destroyed</b> |

## 6 Visualization

## 7 Implementation Details

PyTorch Implementation:

```
# Phase 2: BackFlush with parameter freezing
for name, param in model.named_parameters():
    if 'lm_head' in name or 'output' in name:
        param.requires_grad = False # Freeze phi

for step in range(T):
    h_aux = model.get_embeddings(D_aux)
    L_rot = 1 - F.cosine_similarity(h_aux, -h_aux_cached)
    L_replay = loss_fn(model(D_replay), labels)
    L_total = L_replay + L_rot

    optimizer.zero_grad()
    L_total.backward() # Only updates theta
    optimizer.step()
```

## 8 Summary

1. **Separation:** Backdoors in  $\theta$ , watermarks in  $\phi$  (disjoint)
2. **Correlation:** Phase 1 creates  $h_{backdoor}(\theta) \approx h_{aux}(\theta)$
3. **Confinement:** Freeze  $\phi$ , apply rotation to  $\theta$  only

4. **Result:** Backdoors removed via  $\theta$  updates, watermarks preserved via frozen  $\phi$
5. **GA failure:** Without freezing, updates  $\phi$  and destroys watermarks